



Efficient vision-based vehicle speed estimation

Andrej Macko¹ · Lukáš Gajdošech² · Viktor Kocur²

Received: 20 February 2025 / Accepted: 22 May 2025 / Published online: 14 June 2025
© The Author(s) 2025

Abstract

This paper presents a computationally efficient method for vehicle speed estimation from traffic camera footage. We build upon a previous state-of-the-art method that detects 3D bounding boxes of vehicles using vanishing point geometry and a modified RetinaNet object detector. We show that replacing RetinaNet with a better performing YOLOv6 architecture results in significant improvements in terms of computational efficiency without compromising vehicle speed estimation accuracy. We evaluate the modified method in several variants in terms of vehicle detection and speed estimation accuracy. Our extensive evaluation across various hardware platforms, including edge devices, demonstrates significant gains in frames per second (FPS) compared to the prior state-of-the-art, while maintaining comparable or improved speed estimation accuracy. We analyze the trade-off between accuracy and computational cost, showing that smaller models utilizing post-training quantization offer the best balance for real-world deployment. On the BrnoCompSpeed dataset, our best-performing model achieves better speed estimation accuracy, detection precision and recall than the previous state-of-the-art, while being eight times faster. The transferability of our approach is further confirmed by an evaluation on a different dataset.

Keywords Vehicle speed estimation · Intelligent transportation system · Edge computing · Visual traffic surveillance

Mathematics Subject Classification 68U10

1 Introduction

In today's fast-paced, urbanized world, intelligent transportation systems have become increasingly important in managing the growing complexities of transportation networks. Accurately measuring vehicle speeds is crucial for effective traffic management, speed limit enforcement and the development of intelligent transportation systems. Consequently, there has been growing interest in applying computer vision and machine learning techniques to develop reliable and efficient traffic analysis systems [1].

Processing the vast amount of real-time visual data that cameras generate poses significant computational challenges, as current vision-based traffic surveillance methods rely on deep learning. These computational demands can be addressed in two main ways. The computations can be performed in the cloud, where server-grade hardware can be utilized for efficient computation. The disadvantage of this approach is the need for high bandwidth and complex network infrastructure. As an alternative, edge computing processes sensor data closer to where the data are generated, thereby balancing the computing load and saving network resources. At the same time, edge computing has the potential for improved privacy protection by not transmitting all the raw data to the cloud data centers [2].

When considering vision-based vehicle speed estimation within the edge computing paradigm, it is important to consider the capabilities of edge computing hardware in the design of the methods. Therefore, in this paper, we propose modifications to the state-of-the-art vehicle speed estimation method [3] with a focus on increased computational efficiency. Our key modification is the use of YOLOv6 [4] instead of RetinaNet [5] as the base 2D object detection

✉ Viktor Kocur
viktor.kocur@fmph.uniba.sk

Andrej Macko
macko@photoneo.com

Lukáš Gajdošech
lukas.gajdosech@fmph.uniba.sk

¹ Photoneo, s.r.o., Plynárenská 6, Bratislava, Slovakia

² Faculty of Mathematics, Physics and Informatics, Comenius University Bratislava, Bratislava, Slovakia

network that is modified to detect 3D bounding boxes. We train several variants of vehicle detection networks with varying sizes and evaluate them using six different HW systems with focus on edge computing devices in terms of vehicle speed measurement accuracy and computational costs on the BrnoCompSpeed dataset [6] and the dataset introduced in [7]. We assess the impact of operational precision, model size and image input size on both accuracy and computational efficiency of the networks. We show that the improved accuracy in terms of 2D bounding box localization for larger models does not necessarily translate to improvements in terms of vehicle speed measurement, thus making smaller models a clear choice for vision-based vehicle speed estimation on edge.

The evaluation on the BrnoCompSpeed dataset shows that our best-performing model with quantization achieves slightly better mean (0.71 km/h vs. 0.75 km/h) and median (0.55 km/h vs. 0.58 km/h) speed estimation errors compared to the previous state-of-the-art method [3]. Furthermore, it also has better mean detection precision (91.63% vs. 87.74%) and recall (83.95% vs. 83.21%). Impressively, it manages to be eight times faster, which enables its usage on edge devices, something that was previously unfeasible. We also provide and thoroughly analyze other computationally efficient model configurations, with some models exhibiting significantly better detection precision and recall with similar speed measurement accuracy. We make our code and the trained models publicly available.¹

2 Related works

Vision-based vehicle speed estimation is broadly composed of several steps: traffic camera calibration, vehicle detection and tracking. In this section, we present an overview of the previous literature for individual tasks and overall vehicle speed estimation systems.

2.1 Traffic camera calibration

Traffic camera calibration enables accurate measurements of distances in the road plane by estimating camera intrinsics and extrinsics. Traffic cameras can be calibrated manually using a calibration pattern [8] or distances measured in the road plane [9]. Several semi-automatic methods combine automated approaches with at least one known metric distance in the scene based on vanishing point detection [10–14] or parallel curves [15].

Fully automated methods do not require additional metric information, as they recover the scale automatically. Some methods are based on detection of vanishing points and estimation of the scale based on vehicle dimensions [16, 17]. Other methods rely on detecting landmarks of vehicles [18–20] or their bounding boxes and assuming their known shape [21]. [22] relies on Google Street View to produce a metric 3D model of the traffic scene which can be used to calibrate the camera.

2.2 Computationally efficient object detection

Object detection is a key component of vision-based vehicle speed estimation. Recently, improvements were focused on computational efficiency. The YOLO series [23] represents a staple among efficient object detectors [24, 25]. YOLOv6 [4, 26] built on the previous versions and was specifically optimized for low-power edge deployments by incorporating reparameterized backbones and decoupled detection heads. Alternatives to YOLO [27, 28] also provide object detectors focused on efficiency in low-compute settings.

The performance of object detectors can be further increased with techniques of quantization and knowledge distillation. Quantization-aware training (QAT) and post-training quantization (PTQ) are the two possible approaches to quantization. QAT integrates quantization during training, allowing the model to adjust and mitigate accuracy loss, while PTQ applies quantization after training using calibration data to determine scaling factors and clipping ranges [29]. The second technique, knowledge distillation, transfers knowledge from a large model to a smaller model by minimizing the divergence between their outputs [30], ensuring efficiency while maintaining performance [26, 31]. Post-training quantization strategies have been successfully applied in other real-time detection studies on embedded platforms [32].

2.3 Tracking

Tracking is necessary to associate individual vehicle detections across multiple frames. Common tracking methods include the Kalman filter [33], which is used by SORT [34] and its many variants [35–37] for tracking object bounding boxes. Since modern object detectors provide good accuracy, it is also possible to rely on a simple IoU based tracker [38]. An alternative approach is to perform tracking and detection jointly in a single neural network [39, 40].

2.4 Vision-based vehicle speed estimation

Several pipelines for vehicle speed estimation have been proposed. Most pipelines are composed of camera calibration, vehicle detection and tracking with the final speed obtained

¹ <https://github.com/gajdosech2/Vehicle-Speed-Estimation-YOLOv6-3D>

from the vehicle positions within the track [41]. Vehicle speed estimation was the focus of one of the tracks of the AI City Challenge 2018 [42]. The challenge participants used deep learning-based object detectors in combination with various semi-automatic methods for camera calibration. To obtain the final speeds, the participants used medians, means and percentiles of inter-frame distances or their combinations.

To improve the accuracy of vehicle speed estimation 3D bounding boxes of vehicles can be inferred from known vanishing point geometry and vehicle masks [16, 17]. In [3, 43] 3D bounding boxes are directly estimated by first rectifying the scene based on the known vanishing points and then using a modified 2D object detector which outputs one additional parameter to provide a 3D bounding box. Direct regression of the 3D bounding box via its centroid and vertices is also possible using a specialized neural network [44]. Some works [7, 18] detect license plates instead of the full vehicles to estimate vehicle speeds.

In a completely orthogonal approach, authors of [45] propose a deep neural network which estimates speeds of vehicles directly without previous camera calibration.

2.5 Evaluation datasets

Evaluating vision-based speed estimation methods requires datasets with accurate vehicle ground truth speeds. To provide such data, the authors of the BrnoCompSpeed dataset [6] captured 21 h-long videos taken from a surveillance viewpoint above various roads in the city of Brno. The videos were recorded in 7 different locations. Every road section was recorded using three cameras from different viewing angles. To record the speed of vehicles passing through the surveillance viewpoint, the authors used GPS-synchronized LIDAR thus providing accurate speed estimates.

A smaller dataset was published by Luvizón et al. [7] comprising of 5 h of video of a small section of a three lane road leading up to an intersection. Ground truth speeds of

vehicles are obtained via induction loops installed on the road. This dataset also contains ground truth annotations of vehicle license plates.

The 2018 NVIDIA AI City Challenge [42] included a track for speed estimation from video footage. To evaluate the challenge, the organizers collected a dataset of 27 HD videos, each one minute long. Unfortunately, vehicle speed estimation annotations were not made available publicly, instead requiring challenge participants to use an evaluation server.

3 Computationally efficient vehicle speed estimation

In this section, we provide details about the proposed efficient speed estimation method. The method is based on [3] with several modifications that result in significantly better computational efficiency compared to the baseline. The key modification is the use of a more recent YOLOv6 [4] object detection architecture instead of RetinaNet [5]. We make this modification to significantly decrease the computational demands of the whole speed estimation pipeline to make real-time monocular vehicle speed estimation feasible in low-compute settings.

To make the paper self-contained, we will first provide a brief overview of the method presented in [3] and later provide information on our proposed modifications which result in significantly better computational efficiency.

3.1 Baseline method

The baseline method [3] presents a pipeline (see Fig. 1) for vehicle speed estimation by detecting 3D bounding boxes of vehicles. Unlike traditional 2D bounding boxes, the 3D bounding boxes provide a consistent tracking point at the center of the bottom frontal edge, ensuring reliable speed measurement regardless of the camera angle. This

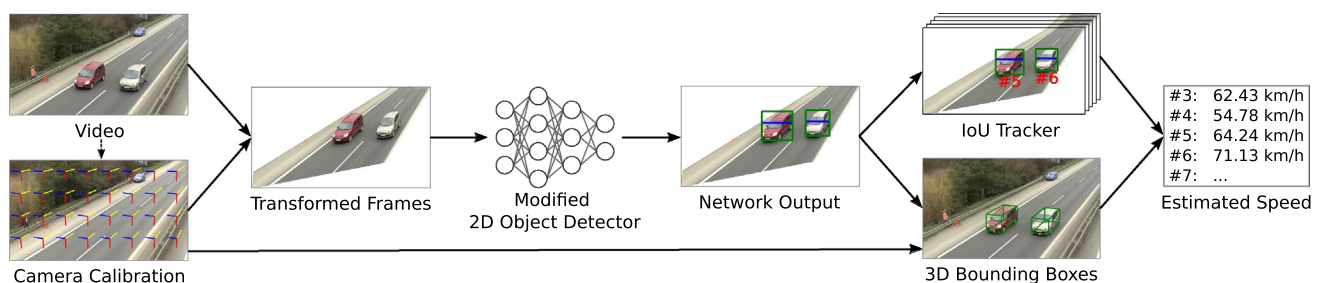


Fig. 1 The vehicle speed estimation pipeline from [3]. During inference, the input images are first transformed using perspective transformation based on prior calibration [17] (see also Fig. 2a-c). A modified 2D object detector is then used to estimate a 2D bounding box of the vehicle along with an additional parameter (see also Fig. 2d).

This information is used to construct a 3D bounding box in the original image. The detected 3D bounding boxes are then tracked using a simple IOU tracker [38]. For each vehicle track, a speed estimate is calculated using the detected vehicle positions and camera calibration [17]

leads to significant improvements in terms of vehicle speed measurement accuracy over a naive approach that relies on 2D bounding boxes.

The pipeline consists of several key steps: camera calibration, image transformation, vehicle detection, 3D bounding box reconstruction, vehicle tracking, and speed estimation.

In the first step, the traffic camera is calibrated using [17]. This calibration method detects vanishing points relevant to the scene and also estimates the scale, enabling accurate metric measurements in the road plane. Based on the obtained vanishing points, a perspective image transformation is constructed such that the directions corresponding with 2 of the 3 relevant vanishing points are aligned with image axes. This rectifies the image, enabling easier detection and at the same time makes it straightforward to parametrize a 3D bounding box of a vehicle as a 2D bounding box with one additional parameter. For a visual representation of this process, see Figure 2. The 2D bounding boxes with the additional parameter are detected using a modified RetinaNet [5]. After detection, the 3D bounding boxes are constructed in the original frame based on the known positions of the vanishing point. Finally, the vehicles are tracked using a simple IoU tracker [38] and their speed is estimated by calculating the median distance traveled between individual frames.

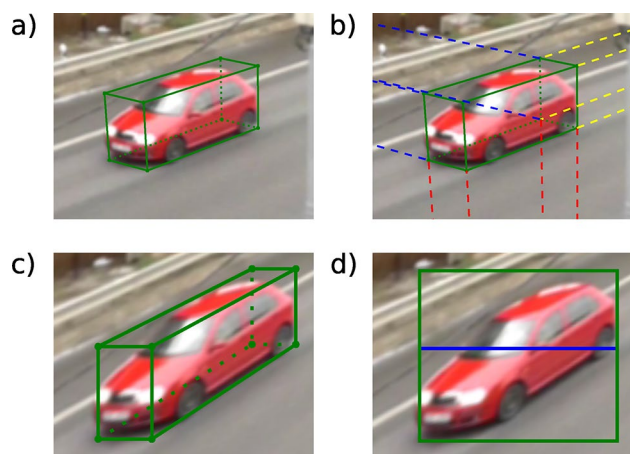


Fig. 2 When vehicles travel on a straight road, their 3D bounding box (a) is aligned with three relevant vanishing points (b). Knowledge of the vanishing points positions, which can be obtained by automatically calibrating the camera (e.g., using [17]), can be used to rectify the image (c). In the rectified image, the task of estimating the 3D bounding box is reduced to finding a 2D bounding box with one additional parameter c_c (d), which determines the position of the top frontal edge (blue) of the 3D bounding box in the 2D bounding box (green). Image adapted from [3]

3.2 Improved base detector

To improve the computational efficiency of the baseline method [3] we propose to use YOLOv6 v3.0 [4] instead of RetinaNet [5]. YOLOv6 is an anchor-free object detector based on a point-based paradigm [46]. The network predicts a 4D output containing the distributions of distances of the object bounding box edges from its center point, classification and objectness output. This output is then converted into final bounding boxes using non-maximum suppression.

The key component of YOLOv6 for fast inference is the RepVGG architecture [47], which leverages structural re-parameterization to optimize performance. During training, RepVGG incorporates multi-branch structures inspired by ResNet [48], including identity and 1×1 branches. After training, these branches are transformed into a single 3×3 convolutional layer using algebraic operations, combining the parameters of the original branches and batch normalization [49]. This results in an inference-time model composed solely of 3×3 convolutions with ReLU, making RepVGG highly efficient on GPUs. YOLOv6 is available in four model sizes: Nano, Small, Medium and Large.

While newer variants such as YOLOv7, YOLOv8, and YOLO-NAS achieve incremental accuracy gains through additional complexities in network design and training strategies [24, 25], their increased computational overhead and implementation intricacies make them less suitable for our focus on deployment rather than absolute performance. Several studies have demonstrated that, for applications where real-time performance and ease of integration are prioritized over peak accuracy, models like YOLOv6 deliver competitive detection capabilities with a favorable accuracy-latency trade-off on embedded platforms [32, 50]. Our goal is to systematically examine the effects of different operational configurations, their influence is expected to be largely independent of the specific detector architecture. Additionally, our chosen revision 3.0 of YOLOv6 includes improvements that put it ahead of newer versions even in raw accuracy.

The standard YOLOv6 architecture produces output classes and 2D bounding boxes of objects. To include the additional parameter c_c introduced in [3] (see Figure 2), we added one convolutional block and one convolution layer for the final prediction to the efficient head. First, the convolutional block comprises a convolution layer with kernel size 3×3 , batch normalization [49], and SiLU [51] activation function. Output of this convolution block is passed to the convolution layer with kernel size 1×1 , and with output size one as we predict the c_c parameter. The modification of a single prediction head is shown in Fig. 3.

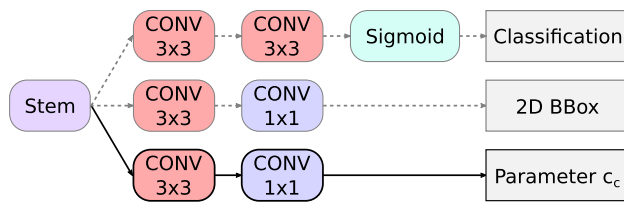


Fig. 3 Our modification of the YOLOv6 [4] prediction head. Processing steps of the original architecture are shown with dashed lines. To obtain the parameter c_c we add a 3×3 convolutional block followed by a 1×1 convolutional block shown with solid lines



Fig. 4 Example of mosaic data augmentation with their respective annotations adjusted to match the new position

3.3 Training data

To train our model, we used two datasets: BrnoCompSpeed [6] and BoxCars116 [52]. The BrnoCompSpeed dataset [6] contains 21 videos that were recorded in 7 different sessions. We used the first four sessions from the BrnoCompSpeed dataset for training and validation. To obtain 3D bounding box annotations we follow the procedure from [3] which combines vehicle masks obtained using [53] and camera calibration data provided with the dataset. This procedure directly provides transformed images and annotations in the form of a 2D bounding box with the additional parameter c_c (see Figure 2). We also use BoxCars116k [6] which contains images of individual vehicles along with camera calibration information and 3D bounding boxes. We use this information to obtain annotations and transformed images.

3.4 Training

During the training process, instead of the preprocessing proposed in [3], we applied mosaic data augmentation. In mosaic augmentation, four images are randomly chosen from the training dataset and combined into a single image by placing them in a 2×2 grid. The new image contains objects from all four original images. The images are additionally flipped and scaled randomly. Their respective annotations (bounding boxes and parameter c_c) are adjusted to match the new positions. Examples of augmented data are shown in Figure 4.

We trained all four available model sizes of the architecture YOLOv6. During training ground truth 2D bounding boxes are assigned to network outputs using TOOD [54]. We incorporate regression of the parameter c_c into the overall loss function using the mean squared error:

$$L_c = \frac{1}{N} \sum_{i=1}^N (c_{c,i}^p - c_{c,i}^g)^2. \quad (1)$$

where $c_{c,i}^p$ is predicted c_c parameter, and $c_{c,i}^g$ is the ground truth value of i -th of the N assigned bounding boxes. The overall loss is calculated as

$$L_{total} = L_{cls} + L_{iou} + L_c, \quad (2)$$

where L_{cls} is the VariFocal classification loss [55], L_{iou} is SIOU [56] (Nano and Small) or GIoU [57] (Medium and Large) for 2D bounding box regression and L_c is the mean-squared error (1) for the c_c parameter.

We trained all our models for 30 epochs, after which there were no further improvements in terms of validation loss. The first three epochs were warm-up epochs. We chose Stochastic Gradient Descent (SGD) with a momentum = 0.937 as the optimization algorithm, a learning rate of 0.01, and a cosine learning rate scheduler. After the full three epochs we selected a snapshot of the model which performed best on the validation set. Based on the superior validation results of the Large model, we also utilized knowledge distillation [31] to distill it to the Nano and Small models [26]. We trained

Table 1 Evaluation of our modified YOLOv6 models the validation set (see Sec. 3.3). mAP and mAR denote mean average precision and recall of the 2D bounding boxes respectively. c_c denotes the mean of error (1) on the validation set

Model	mAP 0.5:0.95	mAR 0.5:0.95	c_c error	Params	FLOPs
Nano	45.0 %	67.4 %	0.0289	4.85 M	11.83 G
Small	51.0 %	72.6 %	0.0267	19.33 M	46.86 G
Medium	51.0 %	71.4 %	0.0219	36.62 M	89.24 G
Large	52.0 %	72.0 %	0.0251	62.71 M	156.36 G
Nano distill	46.0%	67.0%	0.0224	4.85 M	11.83 G
Small distill	50.5%	68.9%	0.0214	19.33 M	46.86 G
Nano quantized	38.9%	59.2%	0.0297	4.85 M	—
Small quantized	44.3%	65.1%	0.0245	19.33 M	—

them for 30 epochs. The first three epochs were warm-up epochs. We chose stochastic gradient descent (SGD) with a momentum = 0.937 as the optimization algorithm, a learning rate of 0.01, and a cosine learning rate scheduler. The results for the trained models on the validation set are provided in Table 1. We note that distillation brought some improvement for the Nano model in terms of mAP, but the distilled counterpart of the Small model performs worse.

3.5 Post-training quantization

For our chosen quantization approach, we opted for PTQ over QAT due to its lower computational cost and simpler deployment process, making it more practical for real-time traffic speed estimation while maintaining accuracy. It follows the pipeline suggested by YOLOv6 authors, where the floating-point ONNX model is converted into a TensorRT v8 engine using its default calibration process [26]. The calibrator runs a representative set of images through the network to collect activation statistics and compute optimal scaling factors that map the FP32 dynamic range to the reduced INT8 space. This mechanism plays a crucial role in maintaining detection accuracy while reducing inference latency and memory footprint.

For the calibration process, we used training data described in Subsection 3.3 with a batch size of 32 across 32 calibration batches. This resulted in processing 1024 training images, allowing us to capture comprehensive statistical properties of activations across all network layers.

4 Results

In this section, we present an evaluation of the trained models with respect to vehicle speed measurement accuracy. We also perform extensive evaluation of the various trained models in terms of computational efficiency.

4.1 Speed measurement evaluation

For vehicle speed measurement evaluation, we use the official evaluation tool² from the authors of [6]. The speed accuracy results for split C of the dataset are shown in Table 2. Among the variants of our proposed method, the Small and Nano models with a 640×360 input resolution provide the best overall performance in both full precision and quantized versions. For better clarity, the speed measurement errors for our best performing models and the best performing prior work [3, 17] are also visualized as box plots in Fig. 5.

Compared to the previous state-of-the-art method from [3] our best performing model (Nano with INT8 quantization and 640×360 input resolution) achieves better results for all evaluated metrics. In the next subsection, we additionally demonstrate that the main advantage of our method lies in its superior computational efficiency.

We note that there are significant variations in terms of recall of the various methods. This may bias the speed measurement accuracy evaluation results as these are calculated from vehicle tracks which are matched correctly to the ground truth. Thus, with higher recall, the set of ground truth vehicle tracks used for evaluation may be larger and include more challenging samples. We therefore include an additional experiment where we evaluate selected models on the same set of vehicle tracks which were properly detected by all methods. The results are shown in Table 3 and confirm the superiority of the Nano quantized model in terms of vehicle speed measurement accuracy.

We also note an interesting observation that the larger models do not perform better than their smaller counterparts despite their better results in terms of mAP and mAR on the validation set (see Table 1). This may occur due to several reasons, such as smaller models generalizing better. Another possibility is that the accuracy of the predicted bounding boxes in individual frames is not as important for the downstream task of vehicle speed estimation, since the speed measurement is aggregated across multiple frames. We also note that an increase in the size of the input image improves the speed measurement accuracy only up to the resolution of 640×360 which may be due to similar reasons.

In addition to the experiments on the BrnoCompSpeed dataset [6] we also evaluated our method on the dataset introduced in [7]. We use the same evaluation metrics as proposed in [7]. Similar to [3] we train a separate model on the training set. Our Small model with input size 640×360 running in F32 operational precision achieves 98.63% recall with 93.95% of estimated speeds falling within the range $(-3, +2)$ km/h of the ground truth speeds. These results are on par with the results reported for [3] which achieved 98.88% recall and 92.71% of estimated speeds within the acceptable speed measurement error range.

4.2 Computational efficiency

For practical uses of vision-based vehicle speed estimation, it is important to also consider the associated computational costs. We therefore perform an evaluation using a variety of HW systems including edge devices, consumer grade PCs and computational servers. The GPUs used for evaluation are listed in Table 4.

When evaluating the computational efficiency of our speed estimation pipeline, it is important to consider the traffic density. Increased traffic density translates into more

² <https://github.com/JakubSochor/BrnoCompSpeed>.

Table 2 Speed measurement evaluation on BrnoCompSpeed test split C [6] with the exception of Learned+RANSAC [21] which was evaluated on the full dataset. The first column indicates operational precision for full precision floats (FP32) and quantized models (INT8). For

the speed measurement error, all methods have the 95% confidence interval in the range ± 0.02 km/h around the mean with the exception of the method by Dubská et al. [16] which has an interval of ± 0.09 km/h

	Model	Input size (px)	Mean error (km/h)	Median error (km/h)	95-th percentile (km/h)	Mean precision (%)	Mean recall (%)
FP32	Dubská et al. [16]	—	8.22	7.87	10.43	73.48	90.08
	SochorAuto [17]	—	1.10	0.97	2.22	90.72	83.34
	SochorManual [17]	—	1.04	0.83	2.35	90.72	83.34
	Learned+RANSAC [21]	—	2.15	1.60	—	—	—
	Transform3D [3]	480 x 270	0.92	0.72	2.35	89.26	79.99
		640 x 360	0.79	0.60	1.96	87.08	83.32
		960 x 540	0.75	0.58	1.84	87.74	83.21
	Nano	480 x 270	0.82	0.66	2.01	91.84	92.54
		640 x 360	0.80	0.64	2.04	92.42	92.57
		960 x 540	0.85	0.66	2.17	92.96	93.09
	Nano distill	480 x 270	1.08	0.82	2.87	89.39	87.96
		640 x 360	0.90	0.73	2.23	90.20	89.60
		960 x 540	0.87	0.70	2.19	89.53	87.19
	Small	480 x 270	0.82	0.64	2.15	95.17	92.99
		640 x 360	0.77	0.59	2.01	93.40	92.48
		960 x 540	0.81	0.63	2.08	94.09	92.23
	Small distill	480 x 270	0.94	0.73	2.42	89.75	88.79
		640 x 360	0.84	0.65	2.16	90.08	89.98
		960 x 540	0.86	0.67	2.16	90.06	90.38
	Medium	480 x 270	0.95	0.78	2.35	92.99	90.82
		640 x 360	0.86	0.69	2.21	93.13	90.95
		960 x 540	0.8	0.66	2.14	93.26	91.32
	Large	480 x 270	0.91	0.75	2.27	92.68	91.19
		640 x 360	0.84	0.66	2.17	92.75	90.94
		960 x 540	0.82	0.64	2.13	92.94	91.13
INT8	Nano	480 x 270	0.73	0.56	1.88	90.95	84.25
		640 x 360	0.71	0.55	1.77	91.63	83.95
		960 x 540	0.81	0.65	2.01	90.73	82.38
	Small	480 x 270	0.78	0.58	2.04	95.98	83.57
		640 x 360	0.80	0.62	2.10	96.24	85.22
		960 x 540	0.83	0.68	2.08	95.38	88.54

The best results are highlighted in bold

Table 3 An alternative evaluation conducted by measuring speed only for vehicles detected by all considered methods. Both Transform3D [3] and our method (Nano variant) are shown using their best-performing input resolutions: 960×540 and 640×360 , respectively

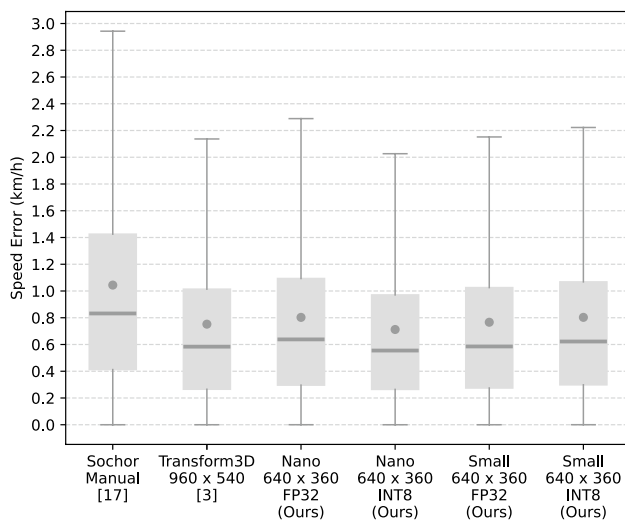
Model	Operational precision	Mean error (km/h)	Median (km/h)
Transform3D [3]	FP32	0.72	0.57
Nano	FP32	0.74	0.61
	FP16	0.74	0.61
	INT8	0.68	0.55

The best results are highlighted in bold

detections per frame and thus increased computational load during the NMS stage of object detection. The three sessions from the test split (sessions 4–6) contain 19.28, 33.52 and 24.38 average cars per minute respectively. To benchmark the FPS across different hardware, we used 10 min of two videos (center and left view) from session 6 in the test set. With 24.38 average cars per minute, this session represents an average traffic density from the test split. For every video, the mean value of FPS was calculated. Finally, after processing both benchmark videos, the overall FPS was calculated as the mean value of the previously calculated mean for the two videos. The FPS estimates include the part of the computation performed on the CPU, which were carried it out

Table 4 Technical parameters of HW systems used in our computational efficiency evaluation

GPU	Memory (GB)	CUDA cores	Tensor cores	FP32 (TFLOPS)	FP16 (TFLOPS)	INT8 (TOPS)	Release year	System CPU
Titan V	12	5120	640	13.80	27.60	55.20	2017	AMD Ryzen 7 2700
RTX 2080	8	2944	368	10.07	20.14	41.20	2018	AMD Ryzen 7 3700X
940 M	2	384	–	0.93	1.83	–	2015	Intel i5 6200U
Jetson TX2	4	256	–	0.66	1.33	–	2016	–
Jetson Xavier	8	384	–	1.41	2.82	21.00	2018	–
Jetson Orin	8	1024	32	1.56	3.13	70.00	2023	–

**Fig. 5** Box plot comparison of our pipeline with YOLOv6 [4, 26] compared to the baseline [3] and the approach by Sochor et al. [17]. We show two of our best performing models (Nano and Small) in F32 and INT8 operational precision variants

in a multi-threaded manner to enable the optimal utilization of computational resources. Table 5 shows the obtained FPS on for the tested HW. The results show significant gains over the previous state-of-the-art method [3] especially for the smaller models. The quantized versions of the two best performing models (Nano and Small with 640x360 input resolution) are capable of running in real-time even on an older edge device (Xavier). This was not possible with the previous approach. We also show that on powerful hardware it is possible to process multiple videos simultaneously.

When considering the speed-vs-accuracy trade-off our evaluation shows the clear superiority of smaller models for the vehicle speed estimation task. Furthermore, quantization and computation in the decreased precision greatly improve computational speed with only a minor decrease in detection recall (see Figure 6).

The Nano model, particularly in its quantized form and on smaller inputs, achieves a very high performance of around 400FPS on desktop GPUs. Based on our experiments, it is not possible to push this performance much further even with stronger hardware, as it is already constrained by the speed of memory access. Therefore, the Nano model is a clear choice for deployment on edge devices, such as Xavier. In other situations, where the retention of precision and recall is important, we advise the usage of the Small quantized model.

We also investigate the effect of traffic density on computational demands. We benchmarked our models in two different situations. We picked a low-density situation by using a one minute long sequence from the test sessions with just 7 cars passing in the view of the camera. On the other hand, a different one minute long sequence with 33 cars was used to represent a situation with high traffic density. See Table 6 for comparison of FPS in these two situations. We provide data only for Titan V, but results for other systems are similar. In the dense traffic conditions the FPS rate drops noticeably, but not more than by 10%.

5 Discussion

In this paper, we have presented a method for computationally efficient vehicle speed estimation. Our method is based on previous work [3] with the base object detector replaced with a modified YOLOv6 [4] architecture. We have presented multiple variants of the proposed method with different model sizes and performed extensive evaluation of the real-world computational requirements using a range of hardware options from edge devices to desktop-grade hardware. By replacing the base object detector with a more efficient architecture we significantly reduce the computational costs while providing similar or better vehicle speed measurement accuracy and improved detection precision and recall. The significantly reduced computational demands

Table 5 Computational efficiency benchmark across various HW systems for different input sizes and data types

Model	Data Type	Input size (px)	940 M (FPS)	RTX 2080 (FPS)	Titan V (FPS)	TX2 (FPS)	Xavier (FPS)	Orin (FPS)
Transform3D [3]	FP32	480 x 270	–	–	70	–	–	–
		640 x 360	–	–	62	–	–	–
		960 x 540	–	–	43	–	–	–
	INT8	480 x 270	–	554	464	–	52	95
		640 x 360	–	355	355	–	38	68
		960 x 540	–	256	246	–	21	57
		480 x 270	49	436	357	19	44	76
		640 x 360	25	311	267	17	35	54
		960 x 540	14	194	179	9	27	38
	FP16	480 x 270	38	295	295	30	38	65
		640 x 360	20	224	221	19	29	43
		960 x 540	10	148	132	12	19	31
Nano	FP32	480 x 270	–	489	392	–	44	126
		640 x 360	–	302	302	–	33	90
		960 x 540	–	205	198	–	16	95
	INT8	480 x 270	23	448	354	15	44	58
		640 x 360	14	243	205	8	36	39
		960 x 540	6	145	139	5	18	24
	FP16	480 x 270	17	229	223	17	28	45
		640 x 360	11	142	142	9	22	30
		960 x 540	4	95	98	5	10	18
Small	FP32	480 x 270	13	356	330	8	36	39
		640 x 360	6	183	174	5	21	24
		960 x 540	2	110	108	2	12	14
	FP16	480 x 270	10	187	162	9	21	31
		640 x 360	5	128	120	5	13	19
		960 x 540	2	55	67	2	5	9
Medium	FP32	480 x 270	–	252	271	–	28	29
		640 x 360	–	143	159	–	17	18
		960 x 540	–	90	84	–	9	10
	FP16	480 x 270	–	130	125	–	15	21
		640 x 360	–	95	95	–	8	14
		960 x 540	–	40	50	–	3	7
Large	FP32	480 x 270	–	–	–	–	–	–
		640 x 360	–	–	–	–	–	–
		960 x 540	–	–	–	–	–	–
	INT8	480 x 270	–	554	464	–	52	95
		640 x 360	–	355	355	–	38	68
		960 x 540	–	256	246	–	21	57
		480 x 270	49	436	357	19	44	76
		640 x 360	25	311	267	17	35	54
		960 x 540	14	194	179	9	27	38
	FP16	480 x 270	38	295	295	30	38	65
		640 x 360	20	224	221	19	29	43
		960 x 540	10	148	132	12	19	31
	FP32	480 x 270	–	489	392	–	44	126
		640 x 360	–	302	302	–	33	90
		960 x 540	–	205	198	–	16	95
	INT8	480 x 270	23	448	354	15	44	58
		640 x 360	14	243	205	8	36	39
		960 x 540	6	145	139	5	18	24
	FP16	480 x 270	17	229	223	17	28	45
		640 x 360	11	142	142	9	22	30
		960 x 540	4	95	98	5	10	18
Small	FP32	480 x 270	13	356	330	8	36	39
		640 x 360	6	183	174	5	21	24
		960 x 540	2	110	108	2	12	14
	FP16	480 x 270	10	187	162	9	21	31
		640 x 360	5	128	120	5	13	19
		960 x 540	2	55	67	2	5	9
Medium	FP32	480 x 270	–	252	271	–	28	29
		640 x 360	–	143	159	–	17	18
		960 x 540	–	90	84	–	9	10
	FP16	480 x 270	–	130	125	–	15	21
		640 x 360	–	95	95	–	8	14
		960 x 540	–	40	50	–	3	7
Large	FP32	480 x 270	–	–	–	–	–	–
		640 x 360	–	–	–	–	–	–
		960 x 540	–	–	–	–	–	–
	INT8	480 x 270	–	554	464	–	52	95
		640 x 360	–	355	355	–	38	68
		960 x 540	–	256	246	–	21	57
		480 x 270	49	436	357	19	44	76
		640 x 360	25	311	267	17	35	54
		960 x 540	14	194	179	9	27	38
	FP16	480 x 270	38	295	295	30	38	65
		640 x 360	20	224	221	19	29	43
		960 x 540	10	148	132	12	19	31
	FP32	480 x 270	–	489	392	–	44	126
		640 x 360	–	302	302	–	33	90
		960 x 540	–	205	198	–	16	95
	INT8	480 x 270	23	448	354	15	44	58
		640 x 360	14	243	205	8	36	39
		960 x 540	6	145	139	5	18	24
	FP16	480 x 270	17	229	223	17	28	45
		640 x 360	11	142	142	9	22	30
		960 x 540	4	95	98	5	10	18

make it possible to deploy our method on edge devices in real-time or on server-grade hardware processing streams from multiple traffic cameras simultaneously.

Our evaluation also shows that minor improvements in bounding box localization accuracy do not necessarily translate to improved vehicle speed estimation accuracy. This may be explained by worse generalization capability of large models and by the fact that speed estimate

is aggregated over multiple frames. Due to this, minor improvements in bounding box localization do not necessarily improve the speed estimate. We also show that computation using lower precision and post-training quantization greatly benefit computational efficiency, while preserving speed measurement accuracy and only marginally reducing detection recall and precision, thus making

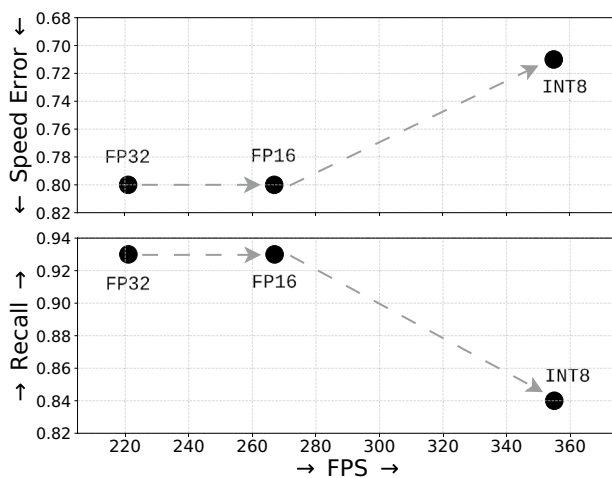


Fig. 6 Quantization improves the performance of all model configurations without significantly affecting speed estimation accuracy. However, this comes at the cost of reduced recall, as quantized models lose the ability to detect all vehicles. Both graphs presents data for an input resolution of 640×360 for Nano model

Table 6 Speed benchmark for different traffic situations. To evaluate computational demands, we selected one minute segments from the test set during which 7 and 33 vehicles passed under the cameras, representing low and high traffic density respectively. The results are shown for the Titan V system

Model	Input size (px)	Low Density (FPS)	High Density (FPS)
Nano	480 x 270	353	319
	640 x 360	297	258
	960 x 540	185	175
Small	480 x 270	351	313
	640 x 360	260	239
	960 x 540	156	145
Medium	480 x 270	288	262
	640 x 360	197	184
	960 x 540	114	108
Large	480 x 270	249	229
	640 x 360	168	159
	960 x 540	89	87

them a clear choice for deploying vision-based vehicle speed estimation systems in practice.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s11554-025-01704-z>.

Acknowledgements Funded by the EU NextGenerationEU through the Recovery and Resilience Plan for Slovakia under the project No. 09I02-03-V01-00029. The authors would also like to thank Photoneo for providing the necessary hardware for evaluation.

Funding Open access funding provided by The Ministry of Education, Science, Research and Sport of the Slovak Republic in cooperation with Centre for Scientific and Technical Information of the Slovak Republic.

Data availability Code for the method and its evaluation as well as the trained models are available at: <https://github.com/gajdosech2/Vehicle-Speed-Estimation-YOLOv6-3D>.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Chen, J., Wang, Q., Cheng, H.H., Peng, W., Xu, W.: IEEE Transactions on Intelligent Transportation Systems **23**(11), 19954 (2022)
- Zhou, X., Ke, R., Yang, H., Liu, C.: Appl. Sci. **11**(20), 9680 (2021)
- Kocur, V., Ftáčnik, M.: Mach. Vision Appl. **31**(7), 1 (2020)
- Li, C., Li, L., Geng, Y., Jiang, H., Cheng, M., Zhang, B., Ke, Z., Xu, X., Chu, X.: arXiv preprint [arXiv:2301.05586](https://arxiv.org/abs/2301.05586) (2023)
- Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P.: Proceedings of the IEEE international conference on computer vision (2017), pp. 2980–2988
- Sochor, J., Juránek, R., Špaňhel, J., Maršík, L., Šíroky, A., Herout, A., Zemčík, P.: IEEE transactions on intelligent transportation systems **20**(5), 1633 (2018)
- Luvizon, D.C., Nassu, B.T., Minetto, R.: IEEE transactions on intelligent transportation systems **18**(6), 1393 (2017)
- He, X.C., Yung, N.H.: Applications of computer vision, 2007. WACV'07. IEEE workshop on (IEEE, 2007), pp. 12–12
- Luvizon, D.C., Nassu, B.T., Minetto, R.: 2014 IEEE international conference on acoustics, speech and signal processing (ICASSP) (IEEE, 2014), pp. 6563–6567
- Maduro, C., Batista, K., Peixoto, P., Batista, J.: Image Processing, 2008. ICIP 2008. 15th IEEE international conference on (IEEE, 2008), pp. 777–780
- You, X., Zheng, Y.: Neurocomputing **204**, 222 (2016)
- Zhang, Z., Tan, T., Huang, K., Wang, Y.: IEEE transactions on circuits and systems for video technology **23**(3), 518 (2013)
- Kocur, V., Ftáčnik, M.: Artificial Neural Networks and Machine Learning - ICANN 2021, pp. 628–639. Springer International Publishing, Cham (2021)
- Li, Y., Zhao, Z., Chen, Y., Zhang, X., Tian, R.: Automatic road-side camera calibration with transformers. Sensors **23**(23), 9527 (2023)
- Corral-Soto, E.R., Elder, J.H.: European conference on computer vision (Springer, 2014), pp. 813–827
- Dubská, M., Herout, A., Sochor, J.: Proceedings of the British machine vision conference, vol. 4 (BMVA Press, 2014), vol. 4, p. 8
- Sochor, J., Juránek, R., Herout, A.: Computer vision and image understanding **161**, 87 (2017)

18. Filipiak, P., Golenko, B., Dolega, C.: European conference on the applications of evolutionary computation (Springer, 2016), pp. 803–818
19. Bhardwaj, R., Tummala, G.K., Ramalingam, G., Ramjee, R., Sinha, P.: ACM transactions on sensor networks (TOSN) **14**(3–4), 1 (2018)
20. Bartl, V., Špaňhel, J., Dobeš, P., Juránek, R., Herout, A.: Machine vision and applications **32**(1), 1 (2021)
21. Revaud, J., Humenberger, M.: Proceedings of the IEEE/CVF international conference on computer vision (2021), pp. 4551–4561
22. Vuong, K., Tamburo, R., Narasimhan, S.G.: Proceedings of the IEEE/CVF winter conference on applications of computer vision (2024), pp. 8553–8562
23. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: Proceedings of the IEEE conference on computer vision and pattern recognition (2016), pp. 779–788
24. Terven, J., Córdova-Esparza, D.M., Romero-González, J.A.: Machine learning and knowledge extraction **5**(4), 1680 (2023). <https://doi.org/10.3390/make5040083>
25. Sapkota, R., Qureshi, R., Calero, M.F., Badjugar, C., Nepal, U., Poullose, A., Zeno, P., Vaddevolu, U.B.P., Khan, S., Shoman, M., Yan, H., Karkee, M.: arXiv (2025). <https://arxiv.org/abs/2406.19407>
26. Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W. et al.: arXiv preprint [arXiv:2209.02976](https://arxiv.org/abs/2209.02976) (2022)
27. Lyu, C., Zhang, W., Huang, H., Zhou, Y., Wang, Y., Liu, Y., Zhang, S., Chen, K.: ArXiv [abs/2212.07784](https://arxiv.org/abs/2212.07784) (2022). <https://api.semanticscholar.org/CorpusID:254685870>
28. Tan, M., Pang, R., Le, Q.V.: 2020 IEEE/CVF Conference on computer vision and pattern recognition (CVPR) pp. 10,778–10,787 (2019). <https://api.semanticscholar.org/CorpusID:208175544>
29. Gholami, A., Kim, S., Dong, Z., Yao, Z., Mahoney, M.W., Keutzer, K.: arXiv preprint [arXiv:2103.13630](https://arxiv.org/abs/2103.13630) (2021)
30. Basterrech, S., Woźniak, M.: 2022 IEEE international conference on systems, man, and cybernetics (SMC) (IEEE, 2022), pp. 3279–3285
31. Li, Z., Xu, P., Chang, X., Yang, L., Zhang, Y., Yao, L., Chen, X.: IEEE transactions on pattern analysis and machine intelligence **45**(8), 10555 (2023). <https://doi.org/10.1109/TPAMI.2023.3257546>
32. Lazarevich, I., Grimaldi, M., Kumar, R., Mitra, S., Khan, S., Sah, S.: 2023 IEEE/CVF international conference on computer vision workshops (ICCVW) pp. 1161–1170 (2023). <https://api.semanticscholar.org/CorpusID:260164672>
33. Kalman, R.E.: J. Basic Eng. **82**(1), 35 (1960)
34. Bewley, A., Ge, Z., Ott, L., Ramos, F., Upcroft, B.: 2016 IEEE international conference on image processing (ICIP) (IEEE, 2016), pp. 3464–3468
35. Wojke, N., Bewley, A., Paulus, D.: 2017 IEEE international conference on image processing (ICIP) (IEEE, 2017), pp. 3645–3649
36. Cao, J., Pang, J., Weng, X., Khirrodar, R., Kitani, K.: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (2023), pp. 9686–9696
37. Du, Y., Zhao, Z., Song, Y., Zhao, Y., Su, F., Gong, T., Meng, H.: IEEE transactions on multimedia **25**, 8725 (2023)
38. Bochinski, E., Eiselein, V., Sikora, T.: 2017 14th IEEE international conference on advanced video and signal based surveillance (AVSS) (IEEE, 2017), pp. 1–6
39. Zhou, X., Koltun, V., Krähenbühl, P.: European conference on computer vision (Springer, 2020), pp. 474–490
40. Bergmann, P., Meinhardt, T., Leal-Taixe, L.: Proceedings of the IEEE/CVF international conference on computer vision (2019), pp. 941–951
41. Zhang, X., Feng, Y., Angeloudis, P., Demiris, Y.: IEEE transactions on intelligent transportation systems **23**(9), 14148 (2022)
42. Naphade, M., Chang, M.C., Sharma, A., Anastasiu, D.C., Jagarlamudi, V., Chakraborty, P., Huang, T., Wang, S., Liu, M.Y., Chellappa, R. et al.: Proceedings of the IEEE conference on computer vision and pattern recognition workshops (2018), pp. 53–60
43. Kocur, V.: Proceedings of the 24th computer vision winter workshop, vol. 2 (2019), vol. 2, pp. 33–41
44. Tang, X., Wang, W., Song, H., Zhao, C.: Complex & Intel. Syst. **9**(4), 4349 (2023)
45. Barros, J., Oliveira, L.: 2021 IEEE intelligent vehicles symposium (IV) (IEEE, 2021), pp. 668–673
46. Tian, Z., Shen, C., Chen, H., He, T.: Proceedings of the IEEE/CVF international conference on computer vision (2019), pp. 9627–9636
47. Ding, X., Zhang, X., Ma, N., Han, J., Ding, G., Sun, J.: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (2021), pp. 13,733–13,742
48. He, K., Zhang, X., Ren, S., Sun, J.: Proceedings of the IEEE conference on computer vision and pattern recognition (2016), pp. 770–778
49. Ioffe, S., Szegedy, C.: International conference on machine learning (pmlr, 2015), pp. 448–456
50. Ling, H., Zhao, T., Zhang, Y., Lei, M.: Appl. Sci. **14**(17) (2024). <https://doi.org/10.3390/app14178054>
51. Elfwing, S., Uchibe, E., Doya, K.: Neural Networks **107**, 3 (2018)
52. Sochor, J., Špaňhel, J., Herout, A.: IEEE transactions on intelligent transportation systems **PP**(99), 1 (2018). <https://doi.org/10.1109/TITS.2018.2799228>
53. He, K., Gkioxari, G., Dollár, P., Girshick, R.: Proceedings of the IEEE international conference on computer vision (2017), pp. 2961–2969
54. Feng, C., Zhong, Y., Gao, Y., Scott, M.R., Huang, W.: in 2021 IEEE/CVF international conference on computer vision (ICCV) (IEEE Computer Society, 2021), pp. 3490–3499
55. Zhang, H., Wang, Y., Dayoub, F., Sunderhauf, N.: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (2021), pp. 8514–8523
56. Gevorgyan, Z.: arXiv preprint [arXiv:2205.12740](https://arxiv.org/abs/2205.12740) (2022)
57. Rezaatofighi, H., Tsoi, N., Gwak, J., Sadeghian, A., Reid, I., Savarese, S.: Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (2019), pp. 658–666

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.